

Evidence půjčovny motokár

Jan Frančák

13. května 2026

Obsah

1	Účel aplikace	2
2	Výběr implementovaných případů užití	2
3	Návrh finální podoby aplikace	2
3.1	Architektura systému	2
4	UML Modely vybraných procesů	3
4.1	Modely tříd	3
4.2	Modely chování	5
5	Závěr	5

1 Účel aplikace

Aplikace slouží jako komplexní informační systém pro správu motokárové arény. Hlavním cílem projektu je digitalizovat a zefektivnit procesy spojené s obsluhou závodníků, evidencí motokár a samotným řízením závodů.

Systém je navržen tak, aby poskytoval přehledné uživatelské rozhraní pro zaměstnance arény, kteří přes něj mohou spravovat databázi závodníků, zakládat nové závody a monitorovat jejich průběh. Aplikace by v reálném nasazení měla být napojena na externí telemetrický systém (RaceControl), který v reálném čase dodává výsledky závodů, jež se následně propisují do uživatelského rozhraní.

Samotný systém RaceControl je mimo scope této práce a jeho funkcionalita je pro potřeby ověření funkcionality mockována (směrem k RaceControl pouze připravená funkce, směrem z RaceControl samostatný python script)

2 Výběr implementovaných případů užití

Pro detailní implementaci a ukázkou funkčnosti systému byly z celkového Use Case modelu vybrány všechny případy užití mimo posledního (5.). Důvodem jejich výběru je skutečnost, že pokrývají základní funkčnosti celé aplikace.

3 Návrh finální podoby aplikace

Aplikace je navržena s využitím moderní vícevrstvé architektury. Rozdělení na frontend a backend zajišťuje dobrou udržitelnost a škálovatelnost systému.

3.1 Architektura systému

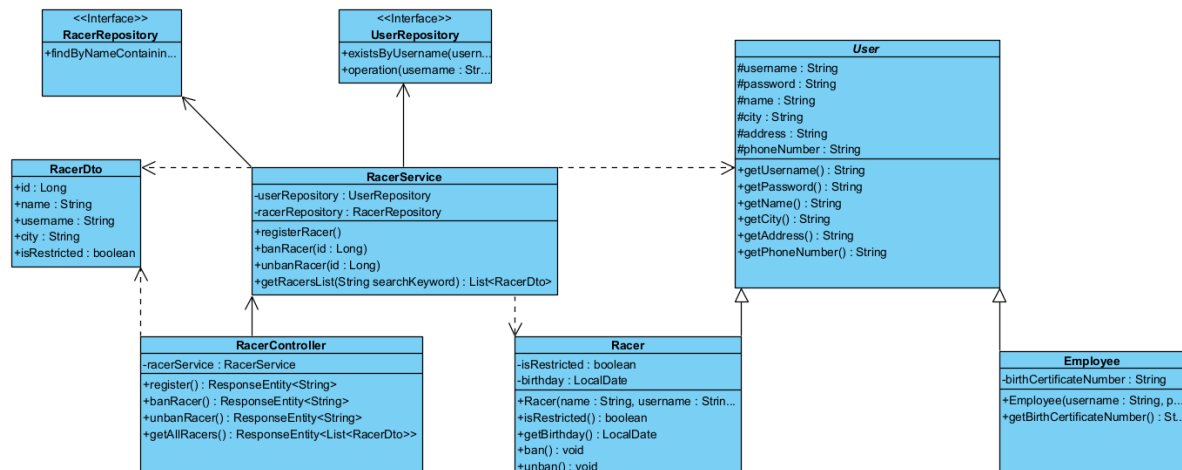
- **Frontend (Uživatelské rozhraní):** Realizován pomocí knihovny React. Komunikuje s backendem pomocí REST API a pro příjem dat v reálném čase využívá technologii Server-Sent Events (SSE).
- **Backend (Aplikační logika):** Postaven na frameworku Spring Boot (Java). Aplikuje vzor (Controller - Service - Repository).
- **Databázová vrstva:** Relační databáze spravovaná pomocí Spring Data JPA a frameworku Hibernate.
- **Externí integrace:** Simulovaný systém RaceControl (v reálném nasazení nezávislá aplikace), se kterým aplikační vrstva komunikuje prostřednictvím dedikovaného `RaceControlClient`.

4 UML Modely vybraných procesů

Pro přesnou definici datových struktur a chování systému byly vypracovány následující UML diagramy, které zachycují realizaci vybraných případů užití.

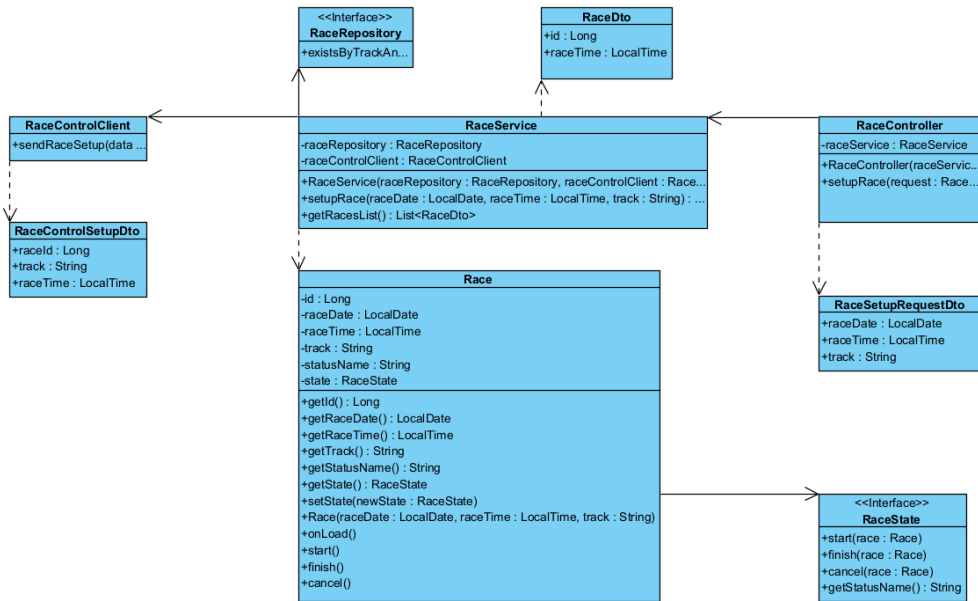
4.1 Modely tříd

Definují strukturu entit a aplikačních rozhraní potřebných pro běh jednotlivých případů užití.



Obrázek 1: Class diagram pro UC-2

Diagram pro UC-2 znázorňuje dědičnost mezi abstraktní třídou **User** a entitou **Racer**. Je zde jasně patrný tok přes vrstvy aplikace: od **RacerController** přes **RacerService** až k repozitářům.



Obrázek 2: Class diagram pro UC-3

Diagram pro UC-3 ukazuje entitu **Race** a využití návrhového vzoru pro stavy (**RaceState**). Zásadní je zde také přítomnost třídy **RaceControlClient**, která by v reálném nasazení obstarávala odesílání požadavků ven ze systému.

4.2 Modely chování

Sekvenční diagramy detailně popisují časovou posloupnost volání metod a komunikaci mezi systémovými vrstvami.

V sekvenčním diagramu UC-2 je detailně namodelován proces vyhledávání pomocí filtru a následné otevření dialogu pro potvrzení akce s využitím UML fragmentu `alt` pro zpracování možností potvrzení nebo zrušení akce.

Sekvenční diagram UC-3 zachycuje proces tvorby závodu. Ústředním bodem je podmíněný blok (`alt`), který v případě kolize trati vrací chybu uživateli, a v případě úspěchu uloží závod do `RaceRepository` a asynchronně informuje `RaceControlClient`.

5 Závěr

Navržená a implementovaná architektura poskytuje základ pro informační systém evidence půjčovny motokár. Myslím si, že by nebylo vůbec nereálné na tomto základu opravdu postavit systém, který by mohl být nasazen v reálném prostředí. Z důvodů silného omezení časem jsem bohužel nestihl dodělat diagram komponent, za což se předem omlouvám.